
Langflow components

Library Hackathon: AI Workflows Without Coding

Langflow 1.12.3 · September 2026

Gefördert durch:



The AI Service Centre Berlin Brandenburg is funded by the Federal Ministry of Research, Technology and Space under the funding code 16IS22092.

This guide describes every component (building block) that participants can use in Langflow during the hackathon: what it does, what goes in and what comes out. The components are listed in the same order as in Langflow's **Components** sidebar on the left: first the categories from top to bottom, then the components in each category in alphabetical order.

It covers Langflow 1.12.3 as it is set up in this kit. The blocks in the *Bibliothek* category were built for the hackathon, all others come with Langflow.

Contents

1. How to read this guide
2. Using an AI model in standard components
3. Components
 - 3.1 Input & Output: Chat Input, Chat Output, Webhook
 - 3.2 Data Sources: API Request, Mock Data, SQL Database, URL, Web Search
 - 3.3 Models & Agents: A2A Agent, Agent, Embedding Model, Language Model, Message History, Prompt Template
 - 3.4 LLM Operations: Batch Run, Guardrails, LLM Selector, Smart Router, Smart Transform, Structured Output
 - 3.5 Files & Knowledge: File System, Knowledge, Memory Base, Read File, Write File
 - 3.6 Processing: Data Operations, Dynamic Create Data, Parser, Split Text, Type Convert
 - 3.7 Flow Control: Human Input, If-Else, Listen, Loop, Notify, Run Flow
 - 3.8 Utilities: Calculator, Current Date, Python Interpreter
 - 3.9 Bibliothek: Katalogsuche, KI-Modell, Literaturangaben prüfen, Literatursuche, Literaturverzeichnis finden, Quellen anreichern, Volltexte holen
4. Not covered here

1. How to read this guide

Inputs and outputs. Inputs are on the **left** edge of a block: fields you fill in, or dots you connect a line to. Outputs are the dots on the **right** edge. A line always runs from an output to an input. Many inputs can be either typed in or connected; once a line is connected, the field shows *Receiving input*.

Data types. Every input and output has a type, and you can only connect matching types. Hover over a dot to see its type.

Type	What it is
Message	Text, for example a chat message, a prompt or a model's answer
JSON	A structured record made of named fields, for example the answer of a web service. Older Langflow versions call it <i>Data</i> .
Table	Rows and columns, like a spreadsheet. Older versions call it <i>DataFrame</i> .
Language Model	A configured AI model that another block can use, for example an Agent
Tool	A block that an Agent may call on its own (see <i>Tool Mode</i> below)
Embeddings	A model that turns text into numbers so similar texts can be found. Only needed for <i>Knowledge</i> .

In the tables below, the type column also names plain settings: *text*, *number*, *switch* (on/off), *choice* (a dropdown or tabs), *slider*, *file* and *list*.

Hidden settings. Many blocks have more settings than they show. Click a block, then click **Parameters** in the small bar above it to show or hide them. This guide lists the hidden settings that are useful; the rest can stay as they are.

Tool Mode. In the same bar above a block there is a **Tool Mode** switch. When it is on, the block gets a *Toolset* output that you connect to an Agent's *Tools* input. The Agent then decides by itself when to use the block. Almost every block can be switched to Tool Mode; the entries below mention it where it is especially useful.

Examples. Where a component is used in one of the example flows in *Starter Project*, the entry says which one, so you can look at it in action.

2. Using an AI model in standard components

Several standard components need an AI model: Agent, Batch Run, Guardrails, LLM Selector, Smart Router, Smart Transform and Structured Output. They have a **Language Model** field with a list of providers such as OpenAI. Those providers are **not set up** in this kit.

Instead, place a **KI-Modell** block (category *Bibliothek*) and connect its **Language Model** output to the component's *Language Model* input. The model then comes from the cluster or from Ollama on your laptop, exactly as set in `.env`. Flow 04 does this for its Agent.

The standard **Language Model** block itself works the same way as KI-Modell but needs a provider; use KI-Modell instead.

3. Components

The components in the order of the sidebar, with one subsection per category.

3.1 Input & Output

3.1.1 Chat Input

Takes the text you type in the **Playground** and passes it into the flow. Almost every flow starts with it.

	Name	Type	What it is
In	Input Text	text	The message. Normally left empty: it is filled from the Playground.
Out	Chat Message	Message	Your message, to connect to a prompt, a model or another block

Hidden settings: *Files* (attach files to the message), *Store Messages* (keep the message in the chat history), *Sender Name*.

Examples: all example flows.

3.1.2 Chat Output

Shows a result in the **Playground**. A flow can have several Chat Outputs; each one appears as its own message.

	Name	Type	What it is
In	Inputs	Message, JSON or Table	What to show. Tables are shown as tables.
Out	Output Message	Message	The same content again, in case you want to pass it on

Hidden settings: *Sender Name* (the name shown above the message, "AI" by default), *Data Template* (how JSON is turned into text).

Examples: all example flows. Flow 01 uses two, one for the report and one for the table.

3.1.3 Webhook

Lets another program start the flow over the internet by sending data to a web address. Not needed during the hackathon.

	Name	Type	What it is
In	Endpoint	text	The address other programs send data to (filled in automatically)
Out	JSON	JSON	The data that was sent

3.2 Data Sources

3.2.1 API Request

Fetches data from a web service (an API) by calling its address. This is how you connect databases that have no ready-made block.

	Name	Type	What it is
In	Mode	choice	<i>URL</i> (fill in address and method) or <i>cURL</i> (paste a ready-made cURL command)
In	URL	text or Message	The address, for example <code>https://api.crossref.org/works</code>
In	Method	choice	<i>GET</i> to fetch data (almost always), <i>POST</i> , <i>PUT</i> , <i>PATCH</i> or <i>DELETE</i> to send data
Out	API Response	JSON	The answer of the service. The actual content is in the field <code>result</code> .

Hidden settings: *Query Parameters* (the search parameters, as a JSON input you can connect), *Headers*, *Body* (for POST), *Timeout* (seconds, 30 by default).

Note: Langflow blocks addresses inside the laptop or the local network, such as `localhost`, with the error *SSRF Protection: ... resolves to blocked IP address*. Public websites and the library databases work.

3.2.2 Mock Data

Creates made-up sample data for trying things out, for example to test a Parser or a Loop before real data is available. It has no inputs.

	Name	Type	What it is
Out	Result	Table, Message or JSON	Sample data in the chosen form

3.2.3 SQL Database

Runs a query against an SQL database. Only useful if you have access to a database, for example a copy of a library system.

	Name	Type	What it is
In	Database URL	text or Message	Where the database is and how to log in, e.g. <code>sqlite:///daten/katalog.db</code>
In	SQL Query	text or Message	The query, e.g. <code>SELECT title FROM books LIMIT 10</code>
Out	Result Table	Table	The result of the query

Tip: in Tool Mode, an Agent can write the SQL queries itself.

3.2.4 URL

Downloads the content of one or more web pages, and optionally of the pages they link to.

	Name	Type	What it is
In	URLs	text or Message	One or more web addresses. Add more with the + button.
In	Depth	slider	1: only the page itself. 2: plus every page it links to. 3: one level further.
Out	Extracted Pages	Table	One row per page, with address and text
Out	Raw Content	Message	The text of all pages together, ready for a prompt

Hidden settings: *Output Format (Text, Markdown or HTML)*, *Prevent Outside* (stay on the same website, on by default), *Headers*, *Timeout*.

Note: the same address check as for *API Request* applies. Some websites, such as Wikipedia, refuse requests without a sender name. Then enter one under *Headers* in the row *User-Agent*, e.g. *Bibliothekshackathon (Langflow)*.

3.2.5 Web Search

Searches the web, news or an RSS feed, without an account or key.

	Name	Type	What it is
In	Search Mode	choice	<i>Web</i> (DuckDuckGo), <i>News</i> (Google News) or <i>RSS</i> (a news feed)
In	Search Query	text or Message	The search words. In <i>RSS</i> mode the field is called <i>RSS Feed URL</i> and takes the address of the feed.
Out	Results	Table	The results with title, link and a text excerpt

Hidden settings: *Max Results* (5 by default), *Max Content Length* (characters kept per result), *Language (hl)* and *Country (gl)*, e.g. *de* and *DE* for German results.

Note: the same address check as for *API Request* applies. **Tip:** a good tool for an Agent.

3.3 Models & Agents

3.3.1 A2A Agent

Sends a message to another agent and returns its answer: either an agent flow in this project (*Internal*) or an agent elsewhere on the internet (*External*). For advanced experiments with several agents working together.

	Name	Type	What it is
In	Mode	choice	<i>Internal</i> (another flow in this project) or <i>External</i> (an address)
In	Agent URL	text or Message	Address of the external agent (only for <i>External</i>)
In	Message	text or Message	What to send to the agent
Out	Response	Message	The agent's answer

3.3.2 Agent

An AI assistant that works on a task step by step and decides by itself which **tools** to use and in which order. The tools are other blocks in Tool Mode, for example *Literatursuche* or *Web Search*.

	Name	Type	What it is
In	Language Model	Language Model	Which AI model the agent thinks with. Connect the <i>Language Model</i> output of KI-Modell here (see above).
In	Agent Instructions	text or Message	The agent's role and rules, e.g. "You are a research assistant in a library. Never invent literature."
In	Tools	Tool	The blocks the agent may use. Several can be connected.
In	Input	text or Message	The task, usually from Chat Input
Out	Response	Message	The agent's final answer
Out	Structured Response	JSON	The answer as structured data (only with an <i>Output Schema</i> , see below)

Hidden settings: *Max Iterations* (how many steps the agent may take, 15 by default), *Current Date* and *Calculator* (built-in tools, on by default), *Number of Chat History Messages* (how much of the conversation the agent remembers), *Output Schema* (a fixed structure for the answer).

Note: agents need models that handle tools well. Small local models often fail at this; use the cluster. In the Playground you can open each step to see which tool the agent used with which input.

Examples: flow 04.

3.3.3 Embedding Model

Turns text into lists of numbers ("embeddings") so that texts with similar meaning can be found. Only needed together with *Knowledge*.

	Name	Type	What it is
In	Embedding Model	choice	The provider and model
Out	Embedding Model	Embeddings	The model, to connect to blocks that need it

Note: this needs an embedding provider, which is not set up in this kit, and KI-Modell can't replace it. Ask the organisers if you want to work with it.

3.3.4 Language Model

Sends a text to an AI model and returns the answer: the standard version of KI-Modell.

	Name	Type	What it is
In	Language Model	choice	Provider and model
In	Input	text or Message	The text for the model, usually a prompt
In	System Message	text or Message	Instructions that apply to the whole conversation
Out	Model Response	Message	The model's answer
Out	Language Model	Language Model	The model itself, for Agent, Batch Run etc.

Hidden settings: *Temperature* (0 = factual and repeatable, 1 = creative), *Max Tokens* (maximum answer length).

Note: the providers are not set up in this kit. Use **KI-Modell** instead: it has the same inputs and outputs.

3.3.5 Message History

Reads earlier chat messages, or stores new ones. This gives a flow a memory of the conversation.

	Name	Type	What it is
In	Mode	choice	<i>Retrieve</i> (read messages) or <i>Store</i> (save a message)
Out	Messages	Message	The earlier messages as text, for a prompt
Out	Table	Table	The earlier messages as a table

Hidden settings: *Number of Messages* (100 by default), *Sender Type* (only the user's, only the AI's, or both), *Order*.

Tip: put the output into a prompt, e.g. Conversation so far: {verlauf}, so the model can refer to earlier questions.

3.3.6 Prompt Template

Writes the text that goes to the AI model. Parts in curly braces, like `{frage}`, are placeholders: for each one the block gets an input of the same name, and the connected text is inserted there.

	Name	Type	What it is
In	Template	text	The instructions with placeholders, e.g. Answer briefly: <code>{frage}</code>
In	<i>one input per placeholder</i>	text or Message	The text that replaces the placeholder
Out	Prompt	Message	The finished text, to connect to a model

Hidden settings: *Use Double Brackets:* use `{{frage}}` instead of `{frage}`, for templates that need single braces as normal text.

Tip: with a lot of data, put the data at the top and the instructions at the end. **Examples:** flows 00 to 03.

3.4 LLM Operations

3.4.1 Batch Run

Runs the AI model once for **every row of a table** and adds the answers as a new column. Useful for summarising or classifying many sources one by one.

	Name	Type	What it is
In	Language Model	Language Model	Connect the <i>Language Model</i> output of KI-Modell here
In	Instructions	text or Message	What to do with each row, e.g. "Summarise this abstract in one sentence."
In	Table	Table	The rows to process, e.g. from <i>Literatursuche</i>
In	Column Name	text	Which column to give to the model. Empty: all columns.
Out	LLM Results	Table	The table with a new column for the answers

Hidden settings: *Output Column Name* (name of the new column, `model_response` by default).

Note: each row is one model call. With a small local model and many rows this takes a long time.

3.4.2 Guardrails

Checks a text for sensitive or unsafe content before it goes on, for example personal data, passwords or attempts to manipulate the AI. The flow then continues on *Pass* or *Fail*.

	Name	Type	What it is
In	Language Model	Language Model	Connect KI-Modell here (needed for the AI checks)
In	Input Text	text or Message	The text to check
In	Guardrails	list	What to check for: <i>PII</i> (personal data), <i>Tokens/Passwords</i> , <i>Jailbreak</i> , <i>Offensive Content</i> , <i>Malicious Code</i> , <i>Prompt Injection</i>
In	Checking Method	choice	<i>AI checks</i> , <i>Rules + AI</i> or <i>Rules only</i> (without a model, but finds less)
Out	Pass	Message	The text, if no problem was found
Out	Fail	Message	The text, if a problem was found
Out	Result Data	JSON	Details of what was found

Tip: a good way to discuss data protection, e.g. before user enquiries are sent to the cluster.

3.4.3 LLM Selector

Chooses the most suitable of several AI models for each input, judged by another model. For comparing models; rarely needed in the hackathon.

	Name	Type	What it is
In	Input	text or Message	The input to answer
In	Language Models	Language Model	The models to choose from (several KI-Modell blocks)
In	Judge LLM	Language Model	The model that decides
In	Optimization	choice	What counts most: <i>quality</i> , <i>speed</i> , <i>cost</i> or <i>balanced</i>
Out	Output	Message	The answer of the chosen model
Out	Selected Model Info	JSON	Which model was chosen
Out	Routing Decision	Message	Why it was chosen

3.4.4 Smart Router

Sorts an input into one of several categories that you define, using the AI model, and sends it on along the matching path. Each category gets its own output.

	Name	Type	What it is
In	Language Model	Language Model	Connect KI-Modell here
In	Input	text or Message	The text to sort, e.g. a user enquiry
In	Routes	table	The categories, one per row, e.g. <i>Ausleihe</i> , <i>Recherche</i> , <i>Sonstiges</i> . Each can have its own output text.
Out	<i>one output per category</i>	Message	The input, on the path of the category it was sorted into

Hidden settings: *Include Else Output* (an extra output for inputs that fit no category), *Additional Instructions* (more rules for sorting).

Tip: for an enquiry service: route loan questions to one flow and research questions to another.

3.4.5 Smart Transform

Filters or reshapes data according to an instruction in plain language, e.g. "keep only publications after 2020". The AI model writes a small program for this, which is then applied to the data.

	Name	Type	What it is
In	Language Model	Language Model	Connect KI-Modell here
In	JSON	JSON, Table or Message	The data to transform
In	Instructions	text or Message	What to do, in your own words
Out	Output	JSON, Table or Message	The transformed data

Note: the result depends on the model. Check it, especially with small local models.

3.4.6 Structured Output

Makes the AI model answer in a fixed structure with named fields, e.g. author, year and title. Ideal for pulling information out of free text.

	Name	Type	What it is
In	Language Model	Language Model	Connect KI-Modell here
In	Input Message	text or Message	The text to extract from
In	Output Schema	table	The fields, one per row: name, description, type (text, number, ...) and whether several values are allowed
Out	Structured Output	JSON or Table	The extracted fields

Hidden settings: *Format Instructions* (the instructions to the model on how to fill the fields).

Tip: e.g. extract author, year, title and journal from each reference in a list, and get a clean table.

3.5 Files & Knowledge

3.5.1 File System

Gives an Agent its own folder in which it may read, create and change files. Only useful in Tool Mode, connected to an Agent.

	Name	Type	What it is
In	Workspace Sub-path	text	A sub-folder within the agent's folder. Empty: the whole folder.
Out	JSON	JSON	Information about the folder

Hidden settings: *Read Only* (the agent may read but not change files).

Note: the folder is inside Langflow's storage in Docker, not in the project folder on your laptop.

3.5.2 Knowledge

Stores texts in a **knowledge base** and searches it later by meaning. This is the basis for "chat with your own documents".

	Name	Type	What it is
In	Mode	choice	<i>Ingest</i> (store texts) or <i>Retrieve</i> (search)
In	Knowledge	choice	Which knowledge base
In	Input	Message, JSON or Table	The texts to store, ideally already split with <i>Split Text</i>
In	Column Configuration	table	Which columns are searched and which are kept as extra information
Out	Results	JSON or Table	What was stored, or the matching texts

Note: knowledge bases need an embedding provider (see *Embedding Model*), which is not set up in this kit. Ask the organisers if you want to try it.

3.5.3 Memory Base

Searches the long-term memory of past conversations with this flow. When *Filter by Session* is off, it searches across all conversations.

	Name	Type	What it is
In	Memory Base	choice	Which memory to search
In	Search Query	text or Message	What to look for
Out	Results	Table	The matching parts of earlier conversations

Hidden settings: *Top K Results* (how many hits, 5 by default), *Filter by Session*.

Note: a Memory Base first has to be created under **Memories** in the left bar. It also needs an embedding provider, which is not set up in this kit.

3.5.4 Read File

Reads an uploaded file and returns its text. Works with PDFs, Word documents, text files, tables and many other formats.

	Name	Type	What it is
In	Files	file	Click Select files and upload one or more files
In	Advanced Parser	switch	Reads complex PDFs, scans and office documents more accurately (with the Docling tool). Much slower and needs a lot of memory.
Out	Raw Content	Message	The text of the file(s)

Hidden settings: *Separator* (what is placed between several files, a blank line by default).

Examples: flow 02 (a master's thesis as a PDF).

3.5.5 Write File

Saves content as a file, e.g. a report as text or a table as Excel.

	Name	Type	What it is
In	File Content	Message, JSON or Table	What to save
In	File Name	text	Name without extension, e.g. pruefbericht
In	File Format	choice	<i>csv, excel, json, markdown, txt or html</i>
In	Append	switch	Add to an existing file instead of replacing it (text formats only)
Out	File Path	Message	Where the file was saved

Note: the file is saved inside Langflow's storage in Docker, not in the project folder. To copy it to the project folder, run `docker compose cp langflow:<file path> .` in a terminal in the project folder, using the path from the output. Often it is easier to show the result in a Chat Output and copy it from there.

3.6 Processing

3.6.1 Data Operations

A toolbox for text, JSON and tables in a single block. First choose what kind of data you have (*Input Type*), then one operation. The matching fields and the output only appear once an operation is chosen.

	Name	Type	What it is
In	Input Type	choice	<i>Text, JSON or Table</i>
In	Operation	choice	One operation, see below
In	Text / JSON / Table	depends on the type	The data to work on
In	<i>more fields</i>		Depending on the operation, e.g. the text to replace or the column to sort by
Out	Message, JSON or Table	depends on the operation	The result

The operations:

- **Text:** *Word Count*, *Case Conversion* (upper/lower case), *Text Replace*, *Text Extract* (find text with a pattern), *Text Head* and *Text Tail* (keep the first or last characters), *Text Strip* (remove spaces or characters at the edges), *Text Join* (join two texts), *Text Clean* (remove empty lines, extra spaces or special characters), *Text to DataFrame* (turn a text table into a real table)
- **JSON:** *Select Keys*, *Remove Keys* and *Rename Keys* (keep, remove or rename fields), *Append or Update* (add fields), *Combine* (merge several records), *Path Selection* (pick one value), *Literal Eval* (turn text into structured data), *JQ Expression* (a small query language for JSON, for everything else)
- **Table:** *Filter* (keep rows that match a condition), *Sort*, *Head* and *Tail* (first or last rows), *Select Columns*, *Drop Column*, *Rename Column*, *Add Column*, *Replace Value*, *Drop Duplicates*, *Merge* and *Concatenate* (combine two tables)

Tip: with *Table* → *Filter* you can e.g. keep only open-access publications from the result of *Literatursuche*.

3.6.2 Dynamic Create Data

Builds a JSON record from fields you define yourself. Each field you add in the configuration becomes an input that you can fill in or connect.

	Name	Type	What it is
In	Input Configuration	table	The fields, one per row: name and type
In	<i>one input per field</i>	depends on the field	The values
Out	JSON	JSON	The record with all fields
Out	Message	Message	The same as text

Tip: e.g. to pass several values (search words, year, number of hits) to *API Request* as query parameters.

3.6.3 Parser

Turns JSON or a table into text using a template. Names in curly braces are replaced by the values of the fields or columns with that name. For tables, the template is applied to each row.

	Name	Type	What it is
In	JSON or Table	JSON or Table	The data
In	Mode	choice	<i>Parser</i> (use the template) or <i>Stringify</i> (turn everything into text as it is)
In	Template	text or Message	e.g. <code>{titel} ({jahr}), DOI {doi}</code>
Out	Parsed Text	Message	The text, one line per row

Hidden settings: *Separator* (what goes between the rows, a line break by default).

Tip: the right step between a table and a prompt, so the model gets exactly the columns it needs.

3.6.4 Split Text

Splits a long text into smaller pieces (chunks). Needed for texts that are too long for the model, or to go through a list piece by piece.

	Name	Type	What it is
In	Input	Message, JSON or Table	The text
In	Separator	text	Where to split: <code>\n</code> for lines, <code>\n\n</code> for paragraphs, <code>.</code> for sentences
In	Chunk Size	number	Maximum length of a piece in characters (1000 by default). Short parts are merged up to this length.
In	Chunk Overlap	number	How many characters neighbouring pieces share (200 by default), so no context is lost at the edges
Out	Chunks	Table	One row per piece, in the column <code>text</code>

Hidden settings: *Clean Output* (only the text column, without extra information from the input), *Keep Separator*.

Tip: to get exactly one piece per line, set *Separator* to `\n`, *Chunk Size* to `1` and *Chunk Overlap* to `0`.

3.6.5 Type Convert

Converts between Message, JSON and Table, for when two blocks don't fit together.

	Name	Type	What it is
In	Input	Message, JSON or Table	The data
In	Output Type	choice	<i>Message</i> , <i>JSON</i> or <i>Table</i>
Out	Message Output / JSON Output / Table Output	depends on the choice	The converted data

Hidden settings: *Auto Parse*: recognises JSON or CSV written as text and turns it into real JSON or a table.

3.7 Flow Control

3.7.1 Human Input

Pauses the flow and asks a person to decide, e.g. to approve or reject a draft. The flow then continues on the output of the chosen answer.

	Name	Type	What it is
In	Input	text or Message	What the person should look at, e.g. the draft
In	User Choices	list	The possible answers, <i>Approve</i> and <i>Reject</i> by default. You can add your own.
Out	<i>one output per answer</i>	Message	The input, on the path of the chosen answer

Tip: for "a human checks before anything is sent", e.g. an AI-written reply to a user enquiry.

3.7.2 If-Else

Compares a text with a value and sends the flow on one of two paths: *True* or *False*. No AI is involved, so the result is always the same.

	Name	Type	What it is
In	Text Input	text or Message	The text to check
In	Operator	choice	How to compare: <i>equals</i> , <i>not equals</i> , <i>contains</i> , <i>starts with</i> , <i>ends with</i> , <i>regex</i> (a search pattern), or for numbers <i>less than</i> , <i>greater than</i> and so on
In	Match Text	text or Message	What to compare with
Out	True	Message	The text, if the condition is met
Out	False	Message	The text, if not

Hidden settings: *Case Sensitive* (on by default), *Case True* and *Case False* (pass a different text instead of the input).

Tip: e.g. only write a detailed report if the check result contains "Abweichung".

3.7.3 Listen

Receives data that a *Notify* block in the same flow has stored under a name, without a connecting line. Marked *Beta* in Langflow.

	Name	Type	What it is
In	Context Key	text or Message	The name the data was stored under
Out	JSON	JSON	The stored data

3.7.4 Loop

Goes through a table or list **item by item**. For each item, the blocks connected to *Item* run once. When everything is done, all results come out together at *Done*.

	Name	Type	What it is
In	Inputs	Table, JSON or Message	The items to go through, e.g. the rows from <i>Split Text</i>
Out	Item	JSON	The current item, for the blocks inside the loop
Out	Done	Table	All results, once every item has been processed

How to close the loop: the last block inside the loop must be connected **back to the *Item* output** of the Loop. That return line tells the Loop that one round is finished. It is shown as a dashed line.

Tip: use a loop when each item needs several steps, e.g. an API request per reference. For one AI call per row, *Batch Run* is simpler.

3.7.5 Notify

Stores data under a name so that a *Listen* block can pick it up elsewhere in the same flow. Marked *Beta* in Langflow.

	Name	Type	What it is
In	Context Key	text	The name to store the data under
In	Input Data	Message, JSON or Table	The data to store
In	Append	switch	Add to data already stored under this name instead of replacing it
Out	JSON	JSON	The stored data

3.7.6 Run Flow

Runs another flow from the same project as a single step. This lets you build small flows and combine them, or give a whole flow to an Agent as a tool. Marked *Beta* in Langflow.

	Name	Type	What it is
In	Flow Name	choice	The flow to run
In	<i>the inputs of that flow</i>		Appear once a flow is chosen
Out	<i>the outputs of that flow</i>		Appear once a flow is chosen

Tip: e.g. give flow 01 to an Agent as a tool, so it can check references as part of a larger task.

3.8 Utilities

3.8.1 Calculator

Calculates an arithmetic expression. Mostly useful as a tool for an Agent, because language models are unreliable at arithmetic.

	Name	Type	What it is
In	Expression	text or Message	e.g. <code>(17 / 60) * 100</code>
Out	JSON	JSON	The result

Note: the Agent has a calculator built in already (hidden setting *Calculator*).

3.8.2 Current Date

Returns the current date and time. Useful in prompts, e.g. for "publications from the last five years".

	Name	Type	What it is
In	Timezone	choice	The time zone, e.g. <code>Europe/Berlin</code> (UTC by default)
Out	Current Date	Message	Date and time as text

3.8.3 Python Interpreter

Runs Python code. For anyone who can program a little; not needed for the challenges.

	Name	Type	What it is
In	Global Imports	text	Python modules the code may use, separated by commas, e.g. <code>math, json</code>
In	Python Code	text or Message	The code. Only what is printed with <code>print()</code> comes out.
Out	Results	JSON	The printed output

3.9 Bibliothek

These components were built for the hackathon. They connect Langflow to library databases and take care of details like splitting reference lists or waiting when a database is busy.

3.9.1 Katalogsuche (hbx / lobid)

Searches for books and other media in the hbx union catalogue (via lobid.org). Good for literature that has no DOI and is therefore missing from Crossref and OpenAlex.

	Name	Type	What it is
In	Suchanfrage (search query)	text or Message	Title, author or keywords, e.g. Umberto Eco Wie man eine wissenschaftliche Abschlussarbeit schreibt
In	Anzahl Treffer (number of hits)	number	How many hits (5 by default)
Out	Treffer (Text)	Message	The hits as a list, for a prompt
Out	Treffer (Tabelle)	Table	The hits as a table

Tip: a good tool for an Agent. **Examples:** flow 04.

3.9.2 KI-Modell

The AI model of the kit. It uses either the cluster (LiteLLM) or Ollama on your laptop, with the address, key and model from `.env`, so nothing has to be entered in the flow.

	Name	Type	What it is
In	Input	text or Message	The text for the model, usually from a Prompt Template
In	System Message	text or Message	Instructions that apply to the whole conversation
In	Quelle (source)	choice	<i>Standard (aus .env) (whatever KI_STANDARD in .env says), Cluster (LiteLLM) or Lokal (Ollama)</i>
In	Modell (model)	choice	Leave empty for the default model. The small arrow button loads the list of available models.
In	Kreativität (Temperatur) (creativity)	slider	0 = factual and repeatable, 1 = creative and varied (0.1 by default)
Out	Model Response	Message	The model's answer
Out	Language Model	Language Model	The model itself, for Agent, Batch Run, Structured Output etc. (see above)

Hidden settings: *Nachdenken erlauben* (let "reasoning" models think before answering: more accurate but much slower), *Maximale Antwortlänge* (maximum answer length), *Zeitlimit* (time limit in seconds, 600 by default), *Adresse* and *API-Schlüssel* (override address and key from `.env`).

Examples: all example flows.

3.9.3 Literaturangaben prüfen (Crossref)

Checks whether the publications in a reference list exist and are cited correctly. Each reference is looked up in Crossref (by DOI or as free text) and compared by title, year and first author. Retracted articles are marked.

	Name	Type	What it is
In	Literaturangaben (references)	text or Message	A reference list: numbered, separated by blank lines, or one reference per line. Lines broken as in a PDF are also recognised.
In	Höchstens prüfen (check at most)	number	Maximum number of references to check (40 by default)
In	Auf zurückgezogene Artikel prüfen (check for retractions)	switch	Also check for retracted articles (on by default)
Out	Prüfbericht (check report)	Message	The result as text: for each reference  confirmed,  discrepancy,  not found or  retracted, with details
Out	Tabelle	Table	The same as a table

Note:  does not always mean "made up": books and websites often have no DOI. Check those with *Katalogsuche*. **Examples:** flows 01 and 04.

3.9.4 Literatursuche (OpenAlex)

Searches for academic publications from all publishers in OpenAlex, with abstracts and links to open-access full texts.

	Name	Type	What it is
In	Suchanfrage (search query)	text or Message	Search words, preferably in English, e.g. large language models academic libraries
In	Anzahl Treffer (number of hits)	number	How many hits (10 by default, at most 50)
In	Erscheinungsjahr ab / bis (year from / to)	number	Limit the years. 0 = no limit.
In	Nur Open Access	switch	Only publications with a free full text
In	Sortierung (sorting)	choice	<i>Relevanz</i> (relevance), <i>Meistzitiert</i> (most cited) or <i>Neueste zuerst</i> (newest first)
Out	Treffer (Tabelle)	Table	The hits as a table, e.g. for <i>Volltexte holen</i>
Out	Trefferliste (Text)	Message	The hits as a list, for a prompt

Hidden settings: *Nur mit Abstract* (only hits with an abstract, on by default).

Note: without an OpenAlex key in `.env`, OpenAlex slows down when many people search; the block then falls back to Crossref for the search. **Examples:** flows 03 and 04.

3.9.5 Literaturverzeichnis finden

Cuts the reference list out of a long document, e.g. a thesis, so that only the references are passed on.

	Name	Type	What it is
In	Dokumenttext (document text)	text or Message	The complete text, e.g. from <i>Read File</i>
Out	Literaturverzeichnis (reference list)	Message	Only the reference list

Examples: flow 02.

3.9.6 Quellen anreichern (OpenAlex)

Adds information from OpenAlex to each reference in a list: abstract, topic, number of citations and open-access status. It also calculates statistics for the whole list, e.g. the share of open-access sources and the range of years.

	Name	Type	What it is
In	Literaturangaben (references)	text or Message	A reference list, e.g. from <i>Literaturverzeichnis finden</i>
In	Höchstens anreichern (enrich at most)	number	Maximum number of references (60 by default)
Out	Überblick (Text) (overview)	Message	Statistics and all sources as text, for a prompt
Out	Quellen (Tabelle) (sources)	Table	All sources with their information, e.g. for <i>Volltexte holen</i>

Examples: flow 02.

3.9.7 Volltexte holen (Open Access)

Downloads the free full texts (PDFs) of publications and extracts their text. If there is no free full text, the abstract is used instead.

	Name	Type	What it is
In	Publikationen (publications)	Table	The table from <i>Literatursuche</i> or <i>Quellen anreichern</i>
In	Höchstens Dokumente (at most documents)	number	How many publications to process (8 by default)
In	Zeichen pro Dokument (characters per document)	number	Longer texts are shortened, keeping beginning and end (3000 by default). Small models can't handle much text.
Out	Texte für die KI (texts for the AI)	Message	The texts, numbered, for a prompt
Out	Tabelle mit Volltexten (table with full texts)	Table	The same as a table

Examples: flow 03.

4. Not covered here

- **Legacy components:** Langflow hides old components that have been replaced by newer ones. They can be shown via the settings icon at the top of the sidebar, but aren't needed.
- **Discover more components / Bundles:** more components from other providers, e.g. OpenAI, Google or vector databases. Most need an account or key and aren't set up in this kit.
- **New Custom Component:** writing your own component in Python. The *Bibliothek* components were made this way.